

CSC456 – Advanced Web Programming

Project 3 (35%)

Project Title: Full-Stack Web Application Development with Authentication and Cloud Deployment.

Description:

In this project, students are required to design, develop, and deploy a **full-stack web application** using **Node.js**, **Express**, **MongoDB**, **Mongoose**, and the **EJS templating engine**. The application must follow the **MVC (Model–View–Controller) architecture** and provide a secure, role-based dashboard for managing several entity types and relationships stored in a MongoDB database.

The main objective of this project is to demonstrate a complete full-stack development workflow, including backend logic, server-side rendering, authentication and authorization, database modeling, deployment, and **Continuous Integration/Continuous Deployment (CI/CD)**.

Core Requirements:

1. Application Architecture

- The application must use **Node.js** and **Express** as the backend framework.
- **MVC** architecture is mandatory:
 - **Models:** **Mongoose** schemas and models for **MongoDB** entities.
 - **Views:** **EJS** templates for all user interfaces.
 - **Controllers:** Application business logic and request handling.
- You are not allowed to use the **Web API** which you have developed in Project 2. However, you can re-use the same **Models** and **MongoDB database** from Project 2.

2. Database & Data Modeling

- Use **Mongoose** for schema definition and database interaction.
- Students must choose their own domain (e.g., school system, inventory management, blog platform, event management).
- Implement:
 - At least three main entity types with full **CRUD** operations.
 - At least one **one-to-many** relationship, and one **many-to-many** relationship between entity types.
- Data validation and schema constraints must be applied using Mongoose.
- You can re-use **Models** and **MongoDB database** from Project 2.

3. Dashboard & Views

- Develop a **server-rendered** dashboard using **EJS** which allows users to manage the different entity types and relationships using a **Web UI**.
- The dashboard must allow authorized users according to their **Roles** (e.g. admin, manager, employee, client...) to:
 - Create new records
 - View lists and details
 - Update existing records
 - Delete records
- Separate views must be implemented for each **CRUD** operation and for managing relationships between entities.
- Reusable layouts and partials are encouraged.

4. Authentication & Authorization

- Implement a complete authentication system, including:
 - **User registration**
 - **User login**
 - **User logout**
- **Password hashing** must be implemented using **bcrypt**.
- **JWT (JSON Web Tokens)** must be used for **session handling** and **user authentication**.
- Implement authorization with roles (e.g., admin, user):
 - Access to routes and dashboard features must be restricted based on user roles.
 - At least two user roles are required.
- Protected routes must not be accessible without valid authentication.

5. Deployment and CI/CD Pipeline

- **Cloud Deployment**
 - The application should be deployed on the cloud (Cloud Deployment) such as:
 - The MongoDB database should be deployed on MongoDB Atlas cloud service (<https://www.mongodb.com/products/platform/atlas-database>).
 - NodeJS/Express application should be deployed on Render cloud service (<https://render.com/>).
 - Environment variables must be handled securely using .env files.
 - The deployed application must be fully functional and accessible online.
- **CI/CD Pipeline**
 - You should implement a basic **Continuous Integration/Continuous Deployment (CI/CD)** pipeline which allows automatic re-deployment every time code is pushed to GitHub. The CI/CD pipeline should be based on the following steps:

1. Code is written or edited locally using VS Code.
2. Modifications are committed to local Git repository.
3. Modifications are pushed to remote Git repository on GitHub.
4. GitHub automatically redeploys app to Render using GitHub actions.

Deliverables:

- A compressed project folder containing:
 - Full Node.js/Express application source code
 - MVC folder structure
 - EJS views
 - Configuration files
 - excluding the *'node_modules'* folder
- A text file containing:
 - Team members' names and IDs.
 - A public GitHub repository link.
 - The live Render deployment URL.
 - The MongoDB Atlas database connection string URI (The connection must be properly configured).
- Students must be able to explain their design choices, architecture, and security implementation during an oral evaluation and show an example of an automatic redeployment after code is edited in VS Code.
- Some Evaluation Criteria:
 - Correct implementation of MVC architecture
 - Functionality of CRUD operations and relationships
 - Quality of authentication and authorization
 - Secure handling of passwords and tokens
 - Proper deployment and CI/CD setup
 - Code organization, readability, and explanation during interview